**МИНОБРНАУКИ РОССИИ**
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**
**Кафедра МО ЭВМ**

# ОТЧЕТ

**по лабораторной работе №4**

**по дисциплине «Построение и анализ алгоритмов»**

**Тема: Поиск подстроки в строке.**

Студент гр. 4343 _____ Едина А.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург

2026

## Задание

**Задание 1**

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T.

**Вход:**

Первая строка – P

Вторая строка – T

Выход:

индексы начал вхождений P в T, разделённые запятой; если P не входит в T, то вывести -1.

**Входные данные**

ab

abab

**Выходные данные**

0,2

**Задание 2**

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B, склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

**Вход:**

Первая строка – P

Вторая строка – T

**Выход:**

Если A является циклическим сдвигом B, то индекс начала строки B в A; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

**Входные данные**

defabc

abcdef

**Выходные данные**

## Выполнение работы.

**1 задание. Файл stepik1.cpp**

Программа считывает две строки: образец p и текст t. Реализует алгоритм Кнута–Морриса–Пратта (КМП) для поиска всех вхождений p в t. Сначала вычисляется префикс-функция для строки combined = p + "#" + t (символ # – разделитель, не встречающийся в исходных строках). Префикс-функция prefic_func (str) – длина наибольшего собственного префикса подстроки combined[0..i], совпадающего с её суффиксом. Для каждой позиции i от length до combined.size()-1 проверяется: если pi[i] == lengthP, то в тексте t найдено вхождение, начинающееся с позиции i - 2·lengthP. Все найденные позиции сохраняются в вектор. Если вхождения найдены, они выводятся через запятую, иначе выводится -1.

**2 задание. Файл stepik2.cpp**

Программа проверяет, является ли строка B циклическим сдвигом строки A. Сначала сравниваются длины: если |A| ≠ |B|, выводится -1. Иначе применяется модифицированный алгоритм КМП. Для строки B вычисляется префикс-функция. Затем выполняется проход по удвоенной циклической строке A (без явного создания A+A). Поддерживается текущее состояние совпадения k. На каждом шаге:

- если текущий символ A[i % lengthA] совпадает с B[k], k увеличивается;
- иначе k уменьшается с помощью префикс-функции.

Если k достигает lengthA, найдено полное совпадение. Позиция начала вхождения pos = i - lengthA + 1. Если pos < lengthA, то это искомый циклический сдвиг, и программа выводит pos (количество сдвигов). В противном случае (если сдвиг не найден за 2·lengthA итераций) выводится -1.

## Исследование

**1 задание (stepik1.cpp)**

**Пространственная сложность**

Алгоритм хранит:

- строку combined = p + "#" + t. длины p.size() + 1 + t.size();
- массив префикс-функции pi того же размера;
- вектор ans для найденных позиций (в худшем случае их количество может быть t.size()).

Обозначим n = p.size(), m = t.size().

Тогда объём используемой памяти: O(n + m) для хранения строк и префикс-функции, плюс O(m) для ответа.

Итоговая сложность по памяти: **O(n + m)**

**Временная сложность**

Вычисление префикс-функции для строки combined выполняется за один проход по всем символам: O(n + m). Внутренний цикл while в совокупности даёт O(n + m), так как каждый шаг уменьшает значение k. Второй проход (поиск вхождений) также выполняется за O(n + m).

Итого: **O(n + m)**

**2 задание (stepik2.cpp)**

**Пространственная сложность**

Алгоритм сохраняет:

- префикс-функцию для строки B (длина n, где n = A.size() = B.size());
- несколько целочисленных переменных.
- Строка A и B уже даны на входе.
- Объём памяти: O(n) для префикс-функции.

Итогов: **O(n)**

**Временная сложность**

Вычисление префикс-функции для B занимает O(n).

Основной цикл выполняется 2n итераций (по удвоенной циклической строке A). На каждой итерации – константное количество операций плюс возможно амортизированное уменьшение k через префикс-функцию (суммарно O(n) за все итерации).

Итого: **O(n)**

## Тестирование

Таблица 1 - Результаты тестирование для задания 1

| Тест | Входные данные | Выходные данные |
|---|---|---|
| 1 | ab<br>ababab | 0,2,4 |
| 2 | aa<br>aaaaa | 0,1,2,3 |
| 3 | abc<br>defg | -1 |
| 4 | abcd<br>abc | -1 |
| 5 | hello<br>hello | 0 |

Таблица 2 - Результаты тестирования для задания 2

| Тест | Входные данные | Выходные данные |
|---|---|---|
| 1 | abc<br>cab | 2 |
| 2 | abc<br>abc | 0 |
| 3 | abc<br>acb | -1 |
| 4 | aaaa<br>aaaa | 0 |
| 5 | abab<br>baba | 1 |

## Приложение

Код из stepik1.cpp:

```
#include <vector>
#include <string>
#include <iostream>

using namespace std;

vector<int> prefic_func(string str){
    vector<int> p(str.size(), 0);
    p.push_back(0);
    for (int i = 1; i < str.size(); i++){
        int k = p[i - 1];
        while (k > 0 && str[i] != str[k]){
            k = p[k - 1];
        }
        if (str[i] == str[k]){
            k++;
        }
        p[i] = k;
    }
    return p;
}


vector<int> kmp(string p, string t){
    int lengthP = p.size();
    int lengthT = t.size();
    vector<int> ans;

    string combined = p + "#" + t;

    vector<int> pi = prefic_func(combined);
    int count = 0;

    for (int i = lengthP; i < combined.size(); i++){
        if (pi[i] == lengthP){
            int pos = i - 2 * lengthP;
            ans.push_back(pos);
        }
    }
    return ans;
}

int main(){
    string p, t;
    cin >> p >> t;
    vector<int> answer = kmp(p, t);
    if (!answer.empty()){

        for (int i = 0; i < answer.size(); i++){
            cout << answer[i];
            if (i != answer.size() - 1){
                cout << ",";
            }
        }
```

```
        cout << endl;
    }else{
        cout << -1 << endl;
    }
}
```

Код из stepik2.cpp:

```
#include <vector>
#include <string>
#include <iostream>

using namespace std;

vector<int> prefic_func(string str){
    vector<int> p(str.size(), 0);
    for (int i = 1; i < str.size(); i++){
        int k = p[i - 1];
        while (k > 0 and str[i] != str[k]){
            k = p[k - 1];
        }
        if (str[i] == str[k]){
            k++;
        }
        p[i] = k;
    }
    return p;
}


int kmp_cycle(string A, string B){
    int lengthB = B.size();
    vector<int> ans;

    vector<int> pi = prefic_func(B);
    int k = 0; // нынешнее состояния совпадения
    int n = A.size();

    for (int i = 0; i < 2*n ; i++)
    {
        // с помощью модульнрой арифметики избегаю создания 2A. abcabc
        // a -> i = 0 -> 0%3 = 0; 1%3=1; 2%3=2; 3%3=0; 4%3=1 ...
        char current_char = A[i % n];

        while (k > 0 && current_char != B[k]){
            k = pi[k - 1];
        }
        if (current_char == B[k]){
            k++;
        }

        if (k == n){
            int pos = i - n + 1;
            if (pos < n) return pos;
            else
                return -1;
        }
    }
```

```
    return -1;
}

int main(){
    string A, B;
    cin >> A >> B;

    if (A.size() != B.size()){
        cout << -1 << endl;
        return 0;
    }

    int answer = kmp_cycle(A, B);

    cout << answer << endl;
    return 0;
}
```